

# Lecture 9: Learning to rank

Huazheng Wang

EECS@OSU

# Probability of relevance

- Three random variables
  - Query  $Q$
  - Document  $D$
  - Relevance  $R \in \{0,1\}$
- Goal: rank  $D$  based on  $P(R=1 | Q,D)$ 
  - Compute  $P(R=1 | Q,D)$
  - Actually, one only needs to compare  $P(R=1 | Q,D_1)$  with  $P(R=1 | Q,D_2)$ , i.e., rank documents
- Several different ways to define  $P(R=1 | Q,D)$

# Conditional models for $P(R=1 | Q,D)$

- Basic idea: relevance depends on how well a query matches a document
  - $P(R=1 | Q,D)=g(\text{Rep}(Q,D),\theta)$  ← a functional form
    - $\text{Rep}(Q,D)$ : feature representation of query-doc pair
      - E.g., #matched terms, highest IDF of a matched term, docLen
  - Using training data (with known relevance judgments) to estimate parameter  $\theta$
  - Apply the model to rank new documents
- Special case: logistic regression

# Regression for ranking?

- Linear regression

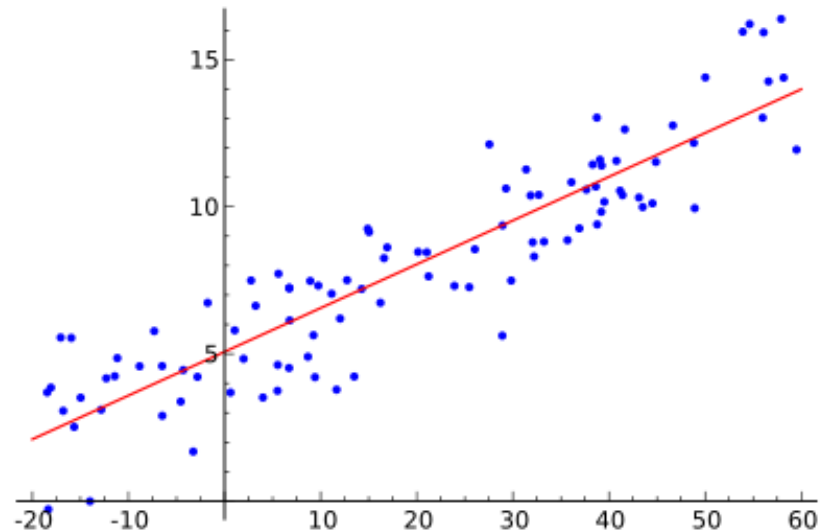
- $y \leftarrow w^T X$

- Relationship between a scalar dependent variable  $y$  and one or more explanatory variables

In a ranking problem:

$X$  features about query-document pair

$y$  relevance label of document for the given query



# Features/Attributes

- Typical features considered in ranking problems

$$X_1 = \frac{1}{M} \sum_1^M \log QAF_{t_j} \quad \text{Average Absolute Query Frequency}$$

$$X_2 = \sqrt{QL} \quad \text{Query Length}$$

$$X_3 = \frac{1}{M} \sum_1^M \log DAF_{t_j} \quad \text{Average Absolute Document Frequency}$$

$$X_4 = \sqrt{DL} \quad \text{Document Length}$$

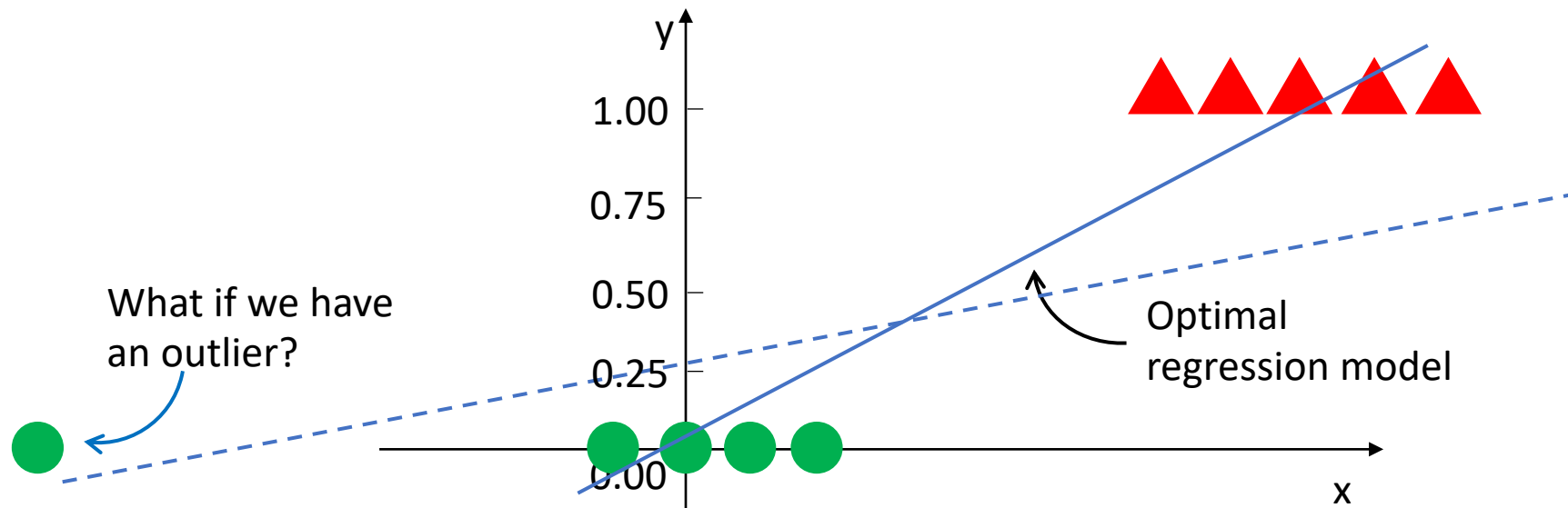
$$X_5 = \frac{1}{M} \sum_1^M \log \frac{N - n_{t_j}}{n_{t_j}} \quad \text{Average Inverse Document Frequency}$$

$$X_6 = \log M \quad \text{Number of Terms in common between query and document}$$

# Regression for ranking

- Linear regression
  - $y \leftarrow w^T X$
  - Relationship between a scalar dependent variable  $y$  and one or more explanatory variables

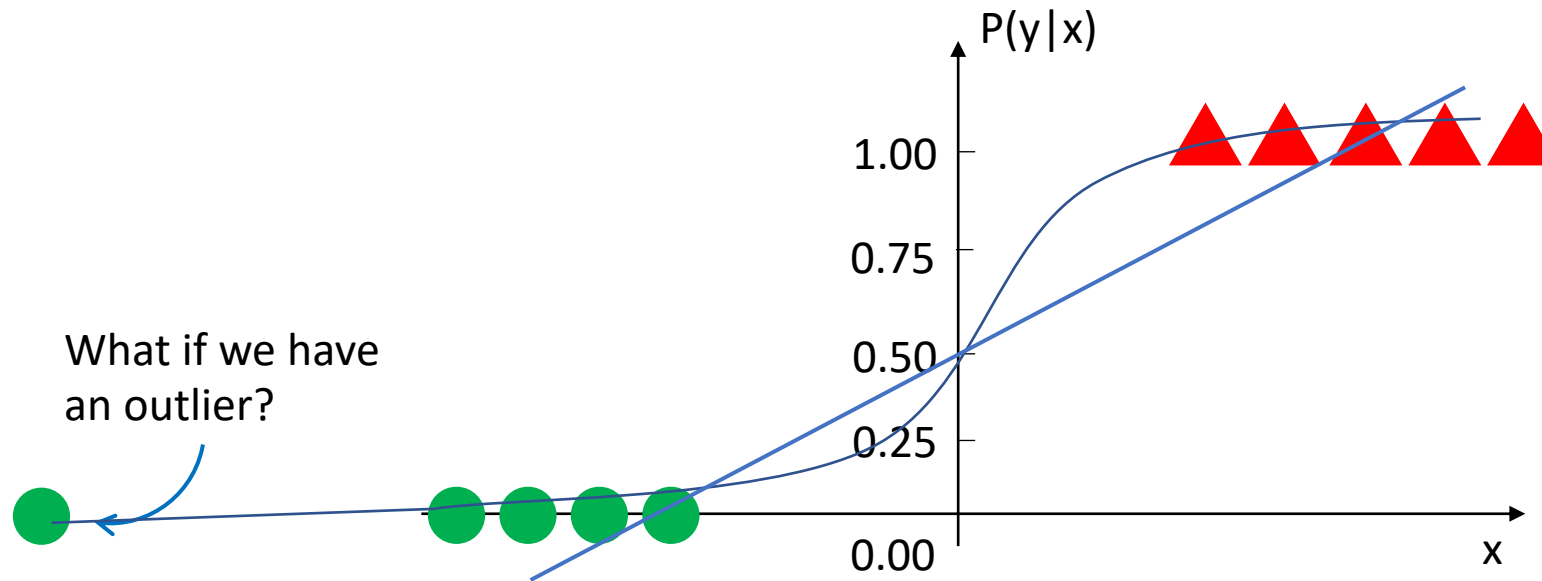
*Y is discrete in a ranking problem!*



# Classification for ranking

- Logistic regression

- $P(R=1|Q,D) = \sigma(w^T X) = \frac{1}{1+\exp(-w^T X)}$
- Directly modeling posterior of document relevance



# Conditional models for $P(R=1 | Q, D)$ Pros & Cons

- Advantages
  - Absolute probability of relevance available
  - May re-use all the past relevance judgments
- Problems
  - Performance heavily depends on the selection of features
  - Little guidance on feature selection
- Will be covered with more details in later learning-to-rank discussions

# Generative models for $P(R=1 | Q, D)$

- Basic idea
  - Compute  $\text{Odd}(R=1 | Q, D)$  using Bayes' rule

$$\text{Odd}(R=1 | Q, D) = \frac{P(R=1 | Q, D)}{P(R=0 | Q, D)} = \frac{P(Q, D | R=1)}{P(Q, D | R=0)} \frac{P(R=1)}{P(R=0)} \leftarrow \text{Ignored for ranking}$$

- Assumption
  - Relevance is a binary variable
- Variants
  - Document “generation”
    - $P(Q, D | R) = P(D | Q, R)P(Q | R)$
  - Query “generation”
    - $P(Q, D | R) = P(Q | D, R)P(D | R)$
    - Language model

# BM25/Okapi approximation

(Robertson et al. 94)

- Idea: model  $p(D | Q, R)$  with a simpler function that approximates 2-Poisson mixture model

- Observations: 
$$\frac{P(R=1 | Q, D)}{P(R=0 | Q, D)} \propto \prod_{i=1, d_i \geq 1}^k \frac{P(A_i = d_i | Q, R=1)P(A_i = 0 | Q, R=0)}{P(A_i = d_i | Q, R=0)P(A_i = 0 | Q, R=1)}$$

- $\log O(R=1 | Q, D)$  is a sum of term weights occurring in both query and document
  - Term weight  $W_i = 0$ , if  $TF_i = 0$
  - $W_i$  increases monotonically with  $TF_i$
  - $W_i$  has an asymptotic limit
- The simple function is 
$$W_i = \frac{TF_i(k_1 + 1)}{K + TF_i} \log \frac{p_i(1 - u_i)}{u_i(1 - p_i)}$$

# Adding document length

- Incorporating doc length
  - Motivation: the 2-Poisson model assumes equal document length
  - Implementation: penalize long doc
  -

$$W_i = \frac{TF_i(k_1 + 1)}{K + TF_i} \log \frac{p_i(1 - u_i)}{u_i(1 - p_i)}$$

where  $K = k_1((1 - b) + b \times \frac{|d|}{\text{avg } |d|})$



*Pivoted document length normalization*

# Adding query TF

- Incorporating query TF
  - Motivation
    - Natural symmetry between document and query
  - Implementation: a similar TF transformation as in document TF

$$W_i = \frac{QTF_i(k_s + 1)}{k_s + QTF_i} \frac{TF_i(k_1 + 1)}{K + TF_i} \log \frac{p_i(1 - u_i)}{u_i(1 - p_i)}$$

- The final formula is called BM25, achieving top TREC performance



***BM: best match***

# The BM25 formula

$$\sum_{T \in Q} w^{(1)} \frac{(k_1 + 1)tf}{K + tf} \frac{(k_3 + 1)qtj}{k_3 + qtj} \quad (1)$$

where

$Q$  is a query, containing terms  $T$

$w^{(1)}$  is the Robertson/Sparck Jones weight [5] of  $T$  in  $Q$

$$\log \frac{(r + 0.5)/(R - r + 0.5)}{(n - r + 0.5)/(N - n - R + r + 0.5)} \quad (2)$$

$N$  is the number of items (documents) in the collection

$n$  is the number of documents containing the term

$R$  is the number of documents known to be relevant to a specific topic

$r$  is the number of relevant documents containing the term

$K$  is  $k_1((1 - b) + b \cdot dl / avdl)$

$k_1$ ,  $b$  and  $k_3$  are parameters which depend on the nature of the queries and possibly on the database;  $k_1$  and  $b$  default to 1.2 and 0.75 respectively, but smaller values of  $b$  are sometimes advantageous; in long queries  $k_3$  is often set to 7 or 1000 (effectively infinite)

$tf$  is the frequency of occurrence of the term within a specific document

$qtj$  is the frequency of the term within the topic from which  $Q$  was derived

$dl$  and  $avdl$  are respectively the document length and average document length measured in some suitable unit.

“Okapi TF/BM25 TF”

becomes IDF when no  
relevance info is available

# The BM25 formula

- A closer look

$$rel(q, D) = \sum_{i=1}^n \overbrace{IDF(q_i) \frac{tf_i(k_1 + 1)}{tf_i + k_1(1 - b + b \frac{|D|}{avg |D|})}}^{TF-IDF \text{ component for document}} \underbrace{\frac{qtf_i(k_2 + 1)}{k_2 + qtf_i}}_{TF \text{ component for query}}$$

- $b$  is usually set to  $[0.75, 1.2]$
- $k_1$  is usually set to  $[1.2, 2.0]$
- $k_2$  is usually set to  $(0, 1000]$

**Vector space model  
with TF-IDF schema!**

# Relevance Estimation

- Query matching
  - Language model
  - BM25
  - Vector space cosine similarity
- Document importance
  - PageRank
  - HITS

# Did I do a good job of ranking documents?

- IR evaluation metrics
  - Precision@K
  - MAP
  - NDCG

# Take advantage of different relevance estimator?

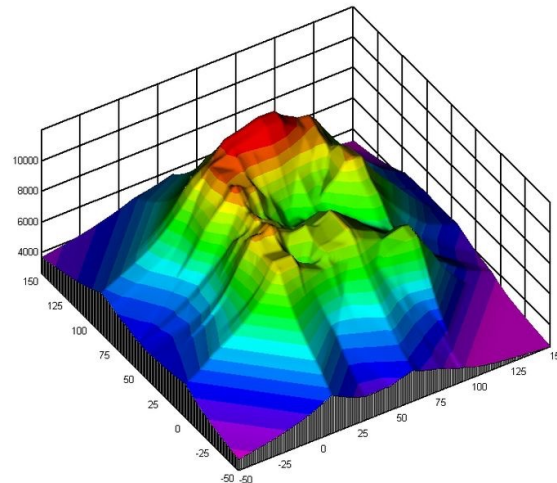
- Ensemble the cues
  - Linear?

$$\alpha_1 \times BM25 + \alpha_2 \times LM + \alpha_3 \times PageRank + \alpha_4 \times HITS$$

$$\{\alpha_1 = 0.4, \alpha_2 = 0.2, \alpha_3 = 0.1, \alpha_4 = 0.1\} \rightarrow \{MAP = 0.20, NDCG = 0.6\}$$

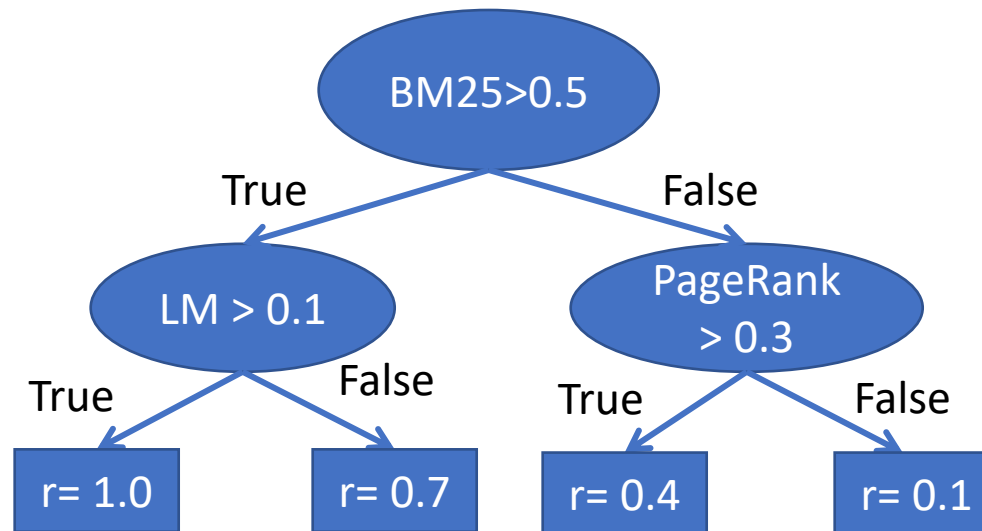
$$\{\alpha_1 = 0.2, \alpha_2 = 0.2, \alpha_3 = 0.3, \alpha_4 = 0.1\} \rightarrow \{MAP = 0.12, NDCG = 0.5\}$$

$$\{\alpha_1 = 0.1, \alpha_2 = 0.1, \alpha_3 = 0.5, \alpha_4 = 0.2\} \rightarrow \{MAP = 0.18, NDCG = 0.7\}$$



# Take advantage of different relevance estimator?

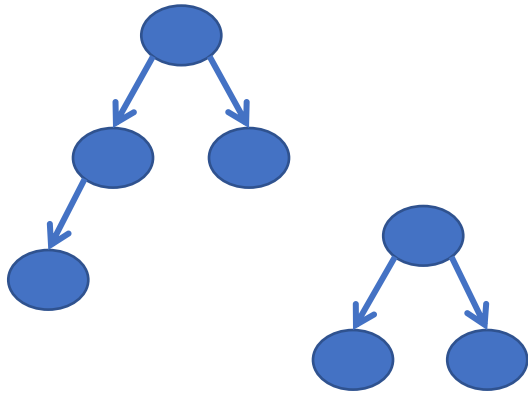
- Ensemble the cues
  - Non-linear?
    - Decision tree-like



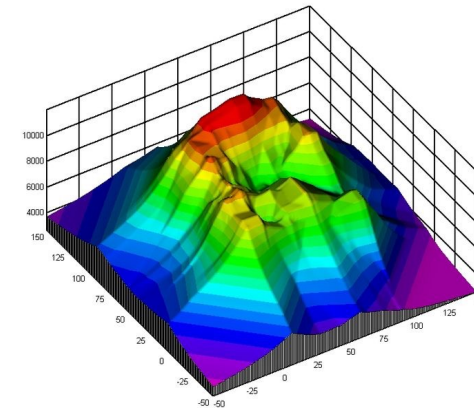
# What if we have thousands of features?

- Is there any way I can do better?
  - Optimize the metrics automatically!

Where to find those tree structures?



How to determine those  $\alpha$ s?



# Rethink the task

- Given: (query, document) pairs represented by a set of relevance estimators, a.k.a., features

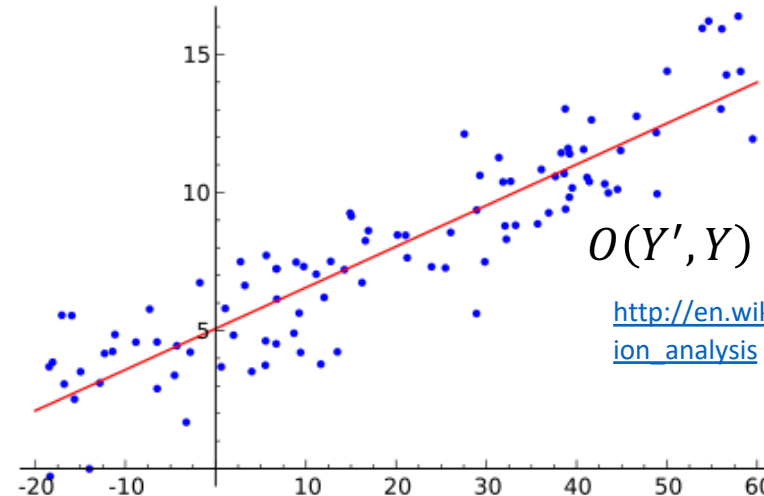
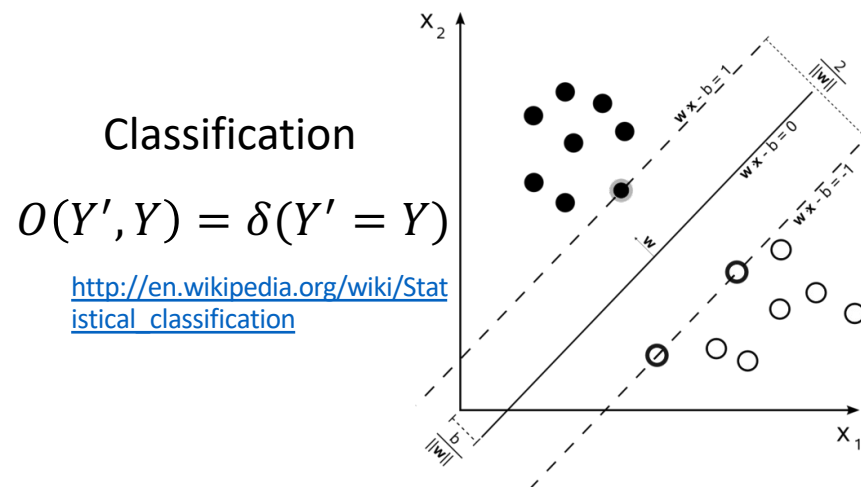
| DocID | BM25 | LM  | PageRank | Label |
|-------|------|-----|----------|-------|
| 0001  | 1.6  | 1.1 | 0.9      | 0     |
| 0002  | 2.7  | 1.9 | 0.2      | 1     |

- Needed: a way of combining the estimators
  - $f(q, \{d\}_{i=1}^D) \rightarrow \text{ordered } \{d\}_{i=1}^D$
- Criterion: optimize IR metrics
  - P@k, MAP, NDCG, etc.

 **Key!**

# Machine Learning

- Input:  $\{(X_1, Y_1), (X_2, Y_2), \dots, (X_n, Y_n)\}$ , where  $X_i \in R^N, Y_i \in R^M$
- Objective function :  $O(Y', Y)$
- Output:  $f(X) \rightarrow Y$ , such that  $f = \operatorname{argmax}_{f' \in F} O(f'(X), Y)$
- Recall: classification/regression



*NOTE: We will only talk about supervised learning.*

# Learning to Rank

- General solution in optimization framework
  - Input:  $\{((q_i, d_1), y_1), ((q_i, d_2), y_2), \dots, ((q_i, d_n), y_n)\}$ , where  $d_n \in R^N, y_i \in \{0, \dots, L\}$
  - **Objective:**  $O = \{P@k, MAP, NDCG\}$
  - Output:  $f(q, d) \rightarrow Y$ , s.t.,  $f = \operatorname{argmax}_{f' \in F} O(f'(q, d), Y)$

| DocID | BM25 | LM  | PageRank | Label |
|-------|------|-----|----------|-------|
| 0001  | 1.6  | 1.1 | 0.9      | 0     |
| 0002  | 2.7  | 1.9 | 0.2      | 1     |

# Challenge: how to optimize?

- Evaluation metric recap

- Average Precision

$$\text{AveP} = \frac{\sum_{k=1}^n (P(k) \times \text{rel}(k))}{\text{number of relevant documents}}$$

- DCG

$$\text{DCG}_p = \text{rel}_1 + \sum_{i=2}^p \frac{\text{rel}_i}{\log_2 i}.$$

*Not continuous with respect to  $f(X)$ !*

- Order is essential!

- $f \rightarrow \text{order} \rightarrow \text{metric}$



**PANIC!**

# Approximate the objective function!

- Pointwise
  - Fit the relevance labels individually
- Pairwise
  - Fit the relative orders
- Listwise
  - Fit the whole order

# Pointwise Learning to Rank

- Ideally perfect relevance prediction leads to perfect ranking
  - $f \rightarrow \mathbf{score} \rightarrow \text{order} \rightarrow \text{metric}$
- Reducing ranking problem to
  - Regression
    - $O(f(Q, D), Y) = -\sum_i ||f(q_i, d_i) - y_i||$
    - Subset Ranking using Regression, D.Cossock and T.Zhang, COLT 2006
  - (multi-)Classification
    - $O(f(Q, D), Y) = \sum_i \delta(f(q_i, d_i) = y_i)$
    - Ranking with Large Margin Principles, A. Shashua and A. Levin, NIPS 2002

# Subset Ranking using Regression

D.Cossock and T.Zhang, COLT 2006

- Fit relevance labels via regression

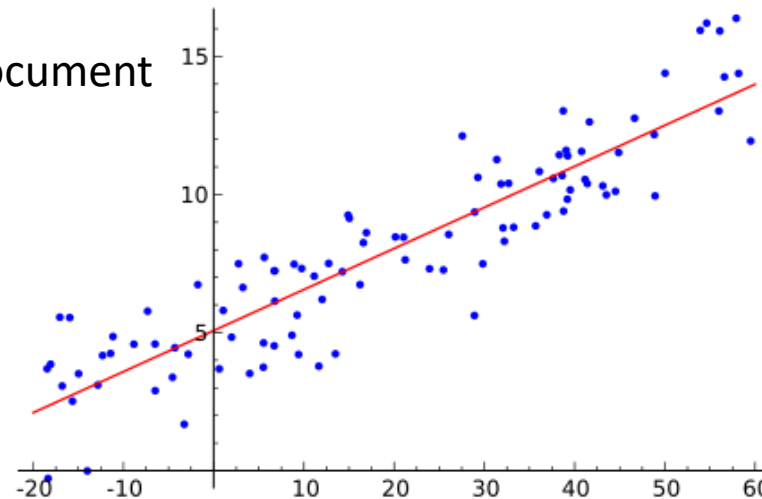
- $$\hat{f} = \arg \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n \left[ \sum_{j=1}^m (f(x_{i,j}, S_i) - y_{i,j})^2 \right]$$

- Emphasize more on relevant documents

- $$\sum_{j=1}^m w(x_j, S) (f(x_j, S) - y_j)^2 + u \sup_j w'(x_j, S) (f(x_j, S) - \delta(x_j, S))^2_+$$

Weights on each document

Most positive document

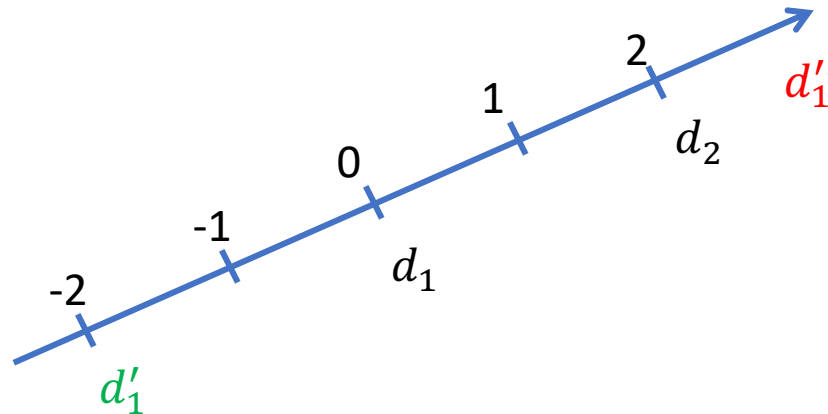


# Pointwise approach

- Machine learning helps!
  - Derive something optimizable
  - More efficient and guided

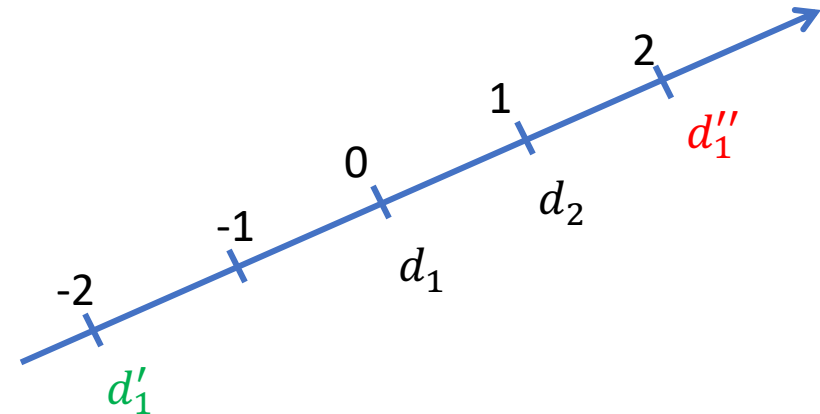
# Deficiencies

- Cannot directly optimize IR metrics
  - $(0 \rightarrow 1, 2 \rightarrow 0)$  worse than  $(0 \rightarrow -2, 2 \rightarrow 4)$
- Position of documents are ignored
  - Penalty on documents at higher positions should be larger
- Favor the queries with more documents



# Pairwise Learning to Rank

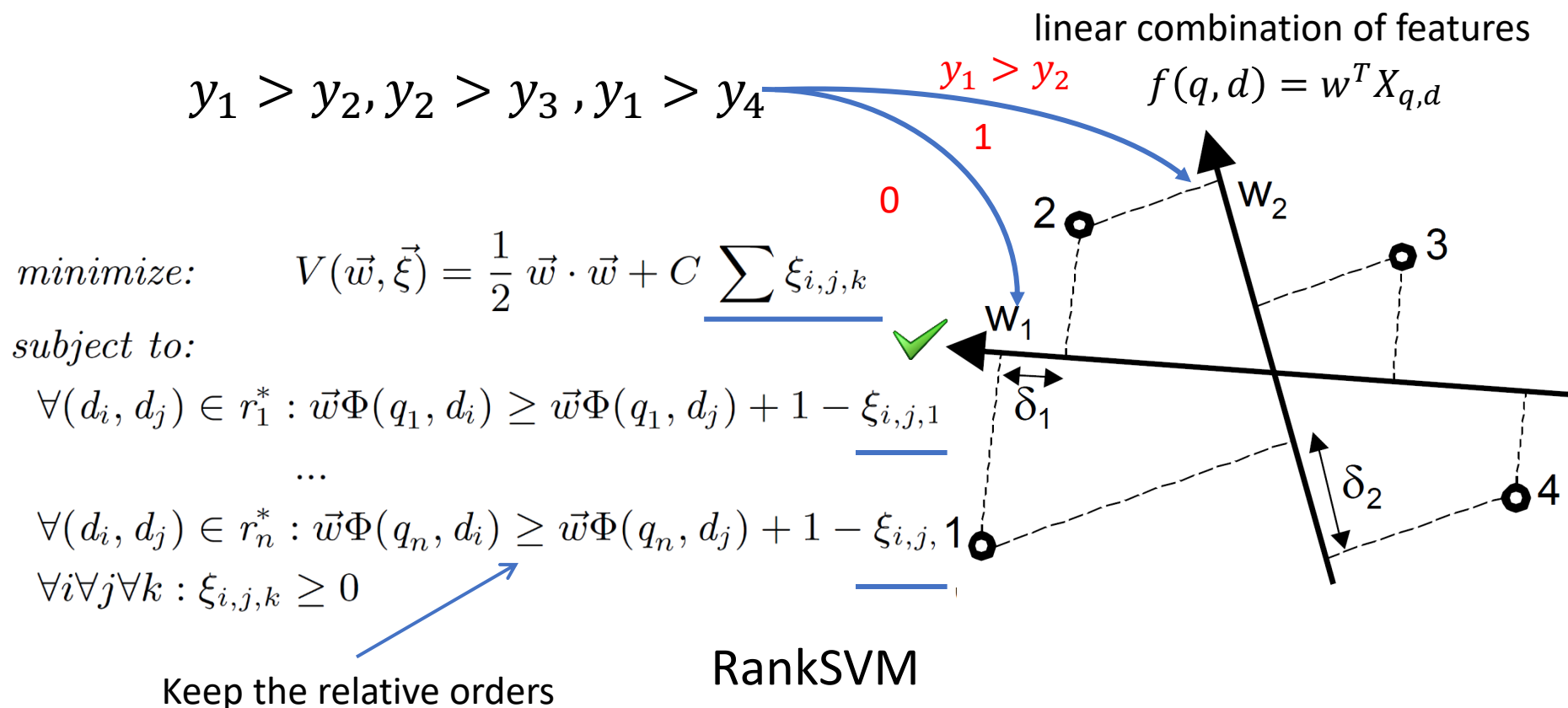
- Ideally perfect partial order leads to perfect ranking
  - $f \rightarrow$  **partial order**  $\rightarrow$  order  $\rightarrow$  metric
- Ordinal regression
  - $O(f(Q, D), Y) = \sum_{i \neq j} \delta(y_i > y_j) \delta(f(q_i, d_i) < f(q_i, d_j))$ 
    - Relative ordering between different documents is significant
    - E.g.,  $(0 \rightarrow -2, 2 \rightarrow 4)$  is better than  $(0 \rightarrow 1, 2 \rightarrow 0)$
- Large body of research



# Optimizing Search Engines using Clickthrough Data

Thorsten Joachims, KDD'02

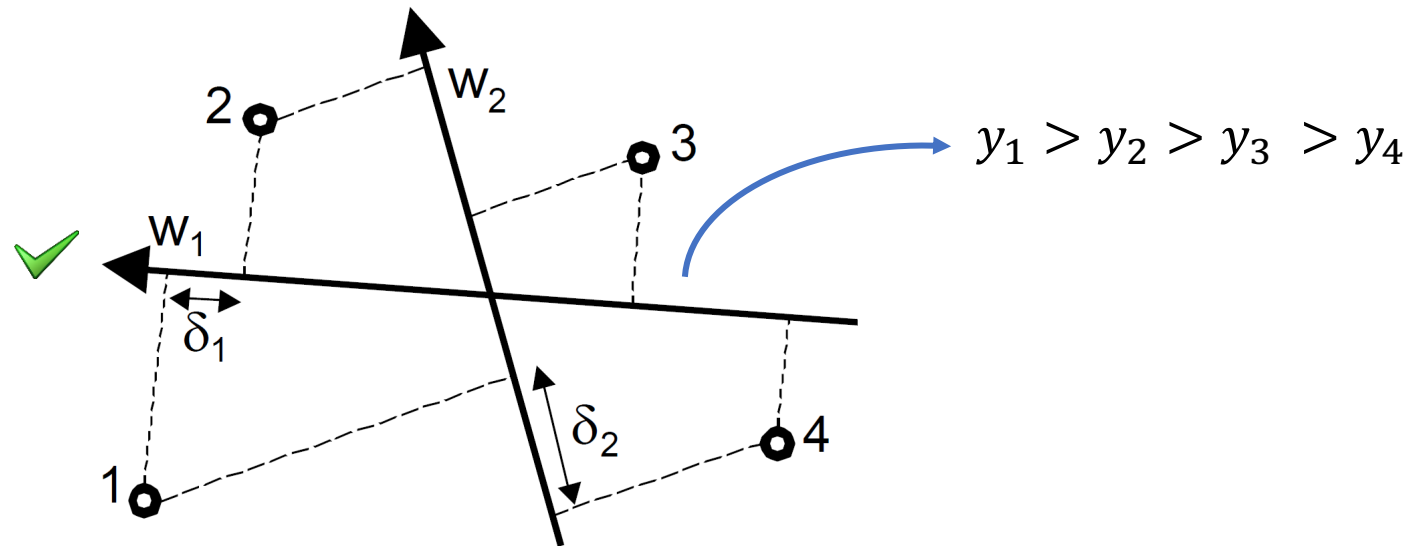
- Minimizing the number of mis-ordered pairs



# Optimizing Search Engines using Clickthrough Data

Thorsten Joachims, KDD'02

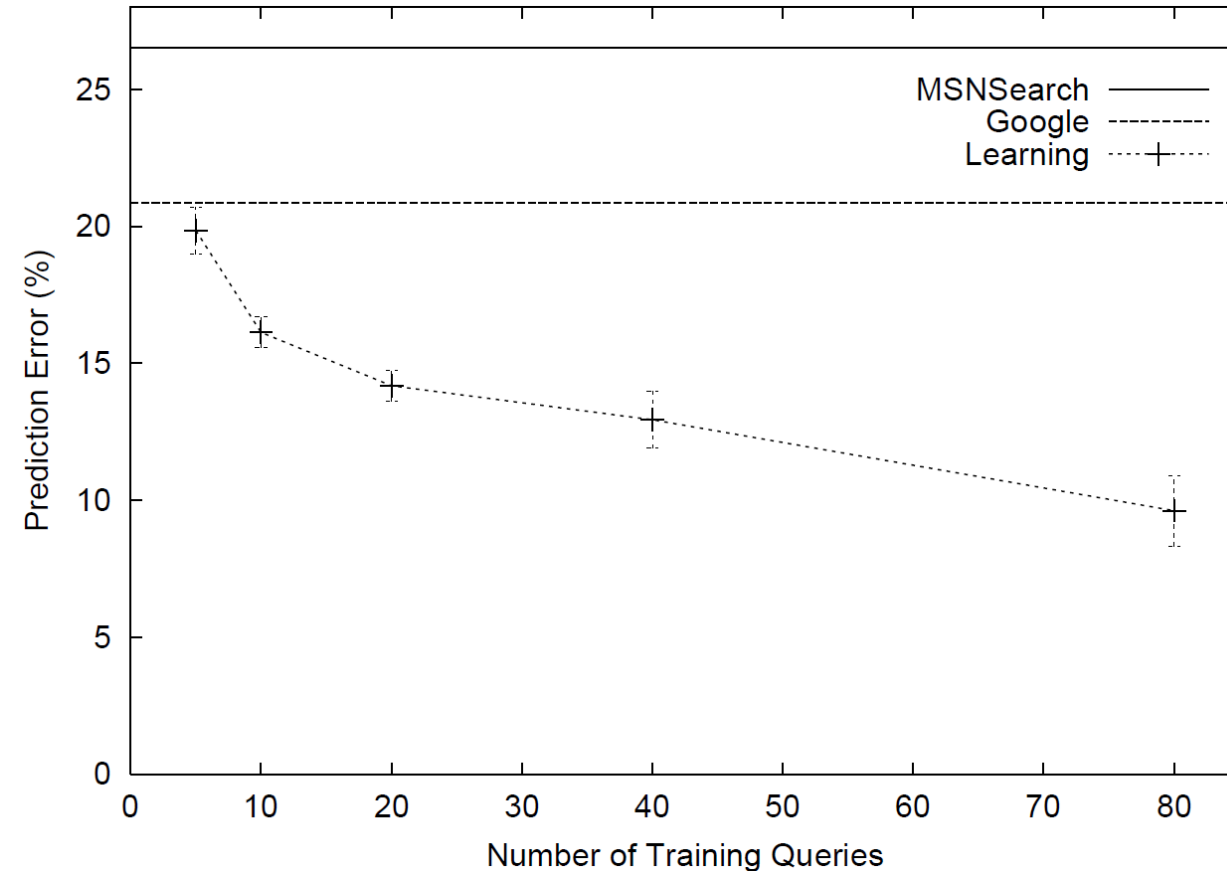
- How to use it?
  - $f \rightarrow \text{score} \rightarrow \text{order}$



# Optimizing Search Engines using Clickthrough Data

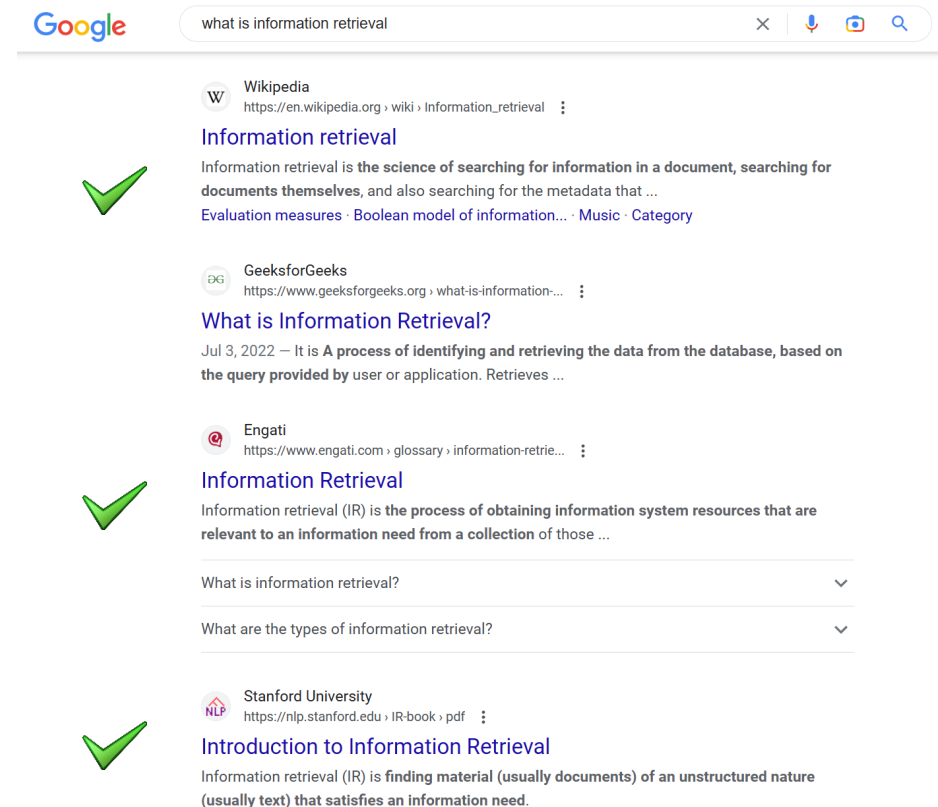
Thorsten Joachims, KDD'02

- How good is it?
  - Test on real system



# Where do we get the relative orders

- Human annotations
  - Small scale, expensive to acquire
- Clickthroughs
  - Large amount, easy to acquire



# Pairwise approach

- Predicting relative order
  - Getting closer to the nature of ranking
- Promising performance in practice
  - Pairwise preferences from click-throughs

# Listwise Learning to Rank

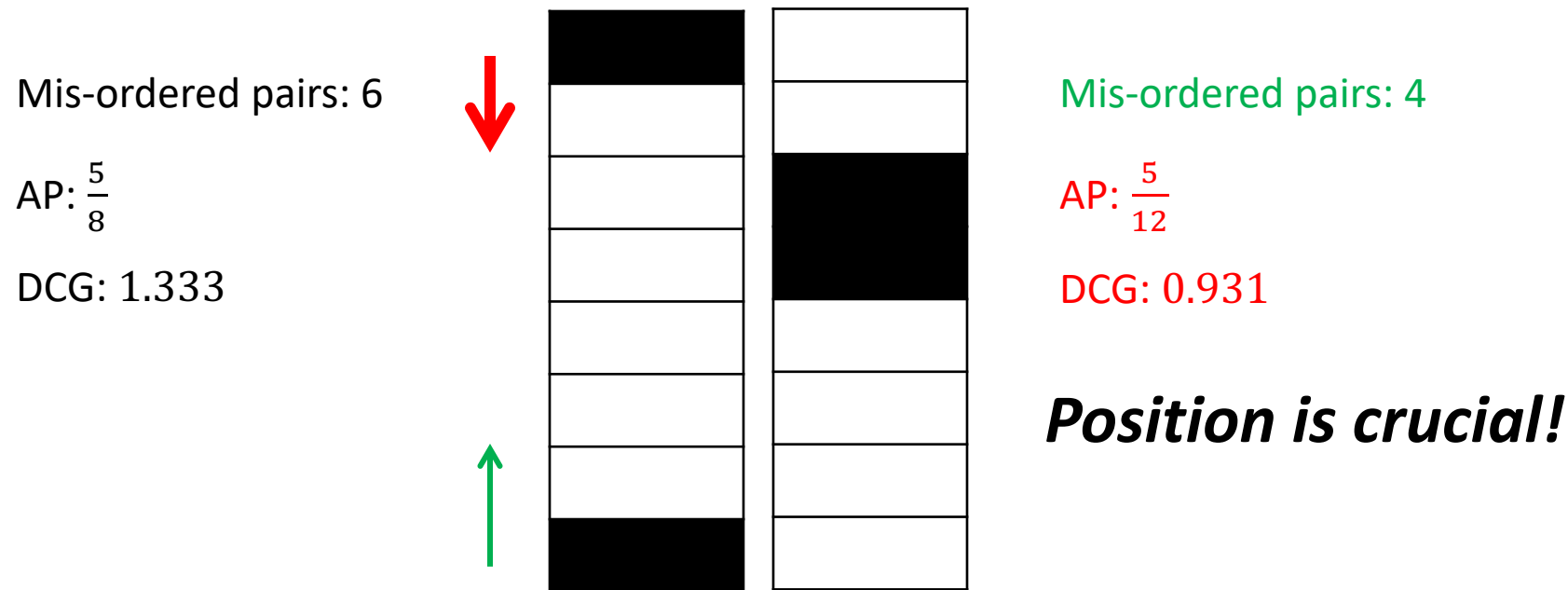
- Can we directly optimize the ranking?
  - $f \rightarrow \text{order} \rightarrow \text{metric}$
- Tackle the challenge
  - Optimization without gradient



# From RankNet to LambdaRank to LambdaMART: An Overview

Christopher J.C. Burges, 2010

- Minimizing mis-ordered pair => maximizing IR metrics?



# From RankNet to LambdaRank to LambdaMART: An Overview

Christopher J.C. Burges, 2010

- Weight the mis-ordered pairs?

- Some pairs are more important to be placed in the right order

- Inject into object function

- $\sum_{y_i > y_j} \Omega(d_i, d_j) \exp(-w \Delta \Phi(q_n, d_i, d_j))$

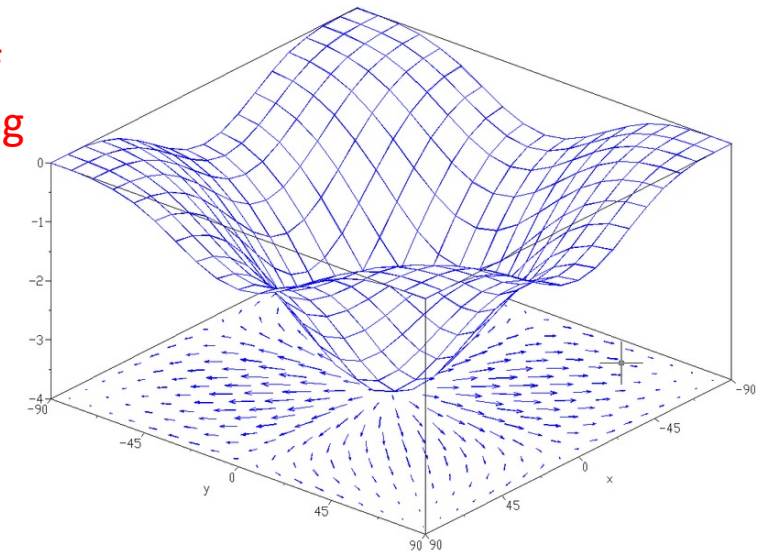
Depend on the ranking of document i, j in the whole list

- Inject into gradient

- $\lambda_{ij} = \frac{\partial O_{appro}}{\partial w} \Delta O$

Change in original object, e.g., NDCG, if we switch the documents i and j, leaving the other documents unchanged

Gradient with respect to approximated objective, i.e., exponential loss on mis-ordered pairs



# From RankNet to LambdaRank to LambdaMART: An Overview

Christopher J.C. Burges, 2010

- Lambda functions
  - Gradient?
    - Yes, it meets the sufficient and necessary condition of being partial derivative
  - Lead to optimal solution of original problem?
    - Empirically

# From RankNet to LambdaRank to LambdaMART: An Overview

Christopher J.C. Burges, 2010

- Evolution

|                                | RankNet                      | LambdaRank                                                              | LambdaMART                                                              |
|--------------------------------|------------------------------|-------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Objective                      | Cross entropy over the pairs | Unknown                                                                 | Unknown                                                                 |
| Gradient ( $\lambda$ function) | Gradient of cross entropy    | Gradient of cross entropy <b>times pairwise change in target metric</b> | Gradient of cross entropy <b>times pairwise change in target metric</b> |
| Optimization method            | neural network               | stochastic gradient descent                                             | Multiple Additive Regression Trees (MART)                               |

Will cover later

Optimize solely by gradient

Non-linear combination

# From RankNet to LambdaRank to LambdaMART: An Overview

Christopher J.C. Burges, 2010

- A Lambda tree

```
<tree id="8" weight="0.1">
  <split>
    <feature> 811 </feature>
    <threshold> 5.0 </threshold>
    <split pos="left">
      <feature> 33 </feature>
      <threshold> 20.0 </threshold>
      <split pos="left">
        <feature> 589 </feature>
        <threshold> 43493.125 </threshold>
        <split pos="left">
          <feature> 1094 </feature>
          <threshold> 302.73438 </threshold>
          <split pos="left">
            <feature> 108 </feature>
            <threshold> 9881.824 </threshold>
            <split pos="left">
              <output> -0.66917753 </output>
            </split>
          <split pos="right">
            <feature> 151 </feature>
            <threshold> 9072276.0 </threshold>
```

splitting

Combination of  
features

# Listwise approach

- Ranking a list of documents as a whole
  - Positions are vital for the ranking algorithms
  - Directly optimizing the target metric
- Limitation
  - The search space is huge!

# Summary

- Learning to rank
  - An automated combination of ranking features for optimizing IR evaluation metrics
- Approaches
  - Pointwise
    - Fit the relevance labels individually
  - Pairwise
    - Fit the relative orders
  - Listwise
    - Fit the whole order

# Experimental Comparisons

- Experiments
  - 1.2k queries, 45.5K documents with 1890 features
  - 800 queries for training, 400 queries for testing

|            | MAP           | P@1           | ERR           | MRR           | NDCG@5        |
|------------|---------------|---------------|---------------|---------------|---------------|
| ListNET    | <i>0.2863</i> | <i>0.2074</i> | <i>0.1661</i> | <i>0.3714</i> | <i>0.2949</i> |
| LambdaMART | <b>0.4644</b> | <b>0.4630</b> | <b>0.2654</b> | <b>0.6105</b> | <b>0.5236</b> |
| RankNET    | 0.3005        | 0.2222        | 0.1873        | 0.3816        | 0.3386        |
| RankBoost  | 0.4548        | 0.4370        | 0.2463        | 0.5829        | 0.4866        |
| RankSVM    | 0.3507        | 0.2370        | 0.1895        | 0.4154        | 0.3585        |
| AdaRank    | 0.4321        | 0.4111        | 0.2307        | 0.5482        | 0.4421        |
| pLogistic  | 0.4519        | 0.3926        | 0.2489        | 0.5535        | 0.4945        |
| Logistic   | 0.4348        | 0.3778        | 0.2410        | 0.5526        | 0.4762        |

# Future

- Tighter bounds
  - Faster solution
  - Larger scale
  - More complex model
  - Wider application scenario
- 
- Brainstorm: LLM for LTR?

# Resources

- Books

- Liu, Tie-Yan. *Learning to rank for information retrieval*. Vol. 13. Springer, 2011.
- Li, Hang. "Learning to rank for information retrieval and natural language processing." *Synthesis Lectures on Human Language Technologies* 4.1 (2011): 1-113.

- Helpful pages

- [http://en.wikipedia.org/wiki/Learning\\_to\\_rank](http://en.wikipedia.org/wiki/Learning_to_rank)

- Packages

- RankingSVM: <http://svmlight.joachims.org/>
- RankLib: <http://people.cs.umass.edu/~vdang/ranklib.html>

- Data sets

- LETOR <http://research.microsoft.com/en-us/um/beijing/projects/letor//>
- Yahoo! Learning to rank challenge <https://webscope.sandbox.yahoo.com/catalog.php?datatype=c>

# References

- Liu, Tie-Yan. "Learning to rank for information retrieval." *Foundations and Trends in Information Retrieval* 3.3 (2009): 225-331.
- Cossock, David, and Tong Zhang. "Subset ranking using regression." *Learning theory* (2006): 605-619.
- Shashua, Amnon, and Anat Levin. "Ranking with large margin principle: Two approaches." *Advances in neural information processing systems* 15 (2003): 937-944.
- Joachims, Thorsten. "Optimizing search engines using clickthrough data." *Proceedings of the eighth ACM SIGKDD*. ACM, 2002.
- Freund, Yoav, et al. "An efficient boosting algorithm for combining preferences." *The Journal of Machine Learning Research* 4 (2003): 933-969.
- Zheng, Zhaohui, et al. "A regression framework for learning ranking functions using relative relevance judgments." *Proceedings of the 30th annual international ACM SIGIR*. ACM, 2007.

# References

- Joachims, Thorsten, et al. "Accurately interpreting clickthrough data as implicit feedback." Proceedings of the 28th annual international ACM SIGIR. ACM, 2005.
- Burges, C. "From ranknet to lambdarank to lambdamart: An overview." Learning 11 (2010): 23-581.
- Xu, Jun, and Hang Li. "AdaRank: a boosting algorithm for information retrieval." Proceedings of the 30th annual international ACM SIGIR. ACM, 2007.
- Yue, Yisong, et al. "A support vector method for optimizing average precision." Proceedings of the 30th annual international ACM SIGIR. ACM, 2007.
- Taylor, Michael, et al. "SoftRank: optimizing non-smooth rank metrics." Proceedings of the international conference WSDM. ACM, 2008.
- Cao, Zhe, et al. "Learning to rank: from pairwise approach to listwise approach." Proceedings of the 24th ICML. ACM, 2007.